

Azure Automated Access Review

Study and Demo Brief. Reference for interview talking points, live demos, and audit conversations. Built for NotebookLM ingestion.

Field	Value
Project	Azure Automated Access Review
Author	Abdie Mohamed
Repository	github.com/abdie-grcengineer/terraform-azure-automated-access-review
Status	Deployed and verified end-to-end (2026-05-06)
Primary framework	NIST 800-53 Rev 5 with CMMC L2 cross-mapping
Target audience	GRC engineers, federal-aligned customers, audit prep

Verification note: federal compliance posture changes monthly. Before any claim in this brief is used in a credibility-sensitive deliverable (agency pitch, resume, certification submission), confirm current status on the Microsoft Service Trust Portal at servicetrust.microsoft.com.

1. The Problem

Traditional GRC programs document controls in Word, evidence them with screenshots, and audit them quarterly. By that point, configuration drift has already happened. The control objective and the actual system state diverge constantly between audits, and the audit cycle is too slow to catch the divergence.

The downstream cost of that gap is well understood: audit prep eats weeks of engineering time per cycle, evidence collection is manual and error-prone, and a control marked 'satisfied' on paper can be quietly broken in production for months before the next sample. For federal customers, that gap directly blocks Authority to Operate (ATO) renewal and FedRAMP continuous monitoring obligations.

GRC Engineering inverts the model. The control becomes the code. Infrastructure as code defines the system, policy as code enforces the rules, and CI runs the gate on every change. There is no separate 'compliance team' reviewing things after the fact. Compliance is implemented in the same pipeline that ships the system, and the audit evidence is generated by construction rather than by a manual collection effort.

2. What This Project Is

A continuous Azure security posture assessment system with policy-as-code guardrails, deployed natively on Microsoft Azure. The system runs on a schedule, pulls findings from native Azure security services, summarizes them with a Microsoft-trained Phi-4-mini-instruct model on Microsoft Foundry, archives a CSV report in Blob Storage, and delivers the report by email via Azure Communication Services. Every infrastructure change is validated against NIST 800-53 / CMMC controls before it touches Azure.

Misconfigurations get blocked at the IaC layer, not at the next quarterly audit.

Tech stack

Layer	Choice
Infrastructure as code	Terraform 1.10+, hashicorp/azurerm v4
State management	Azure Storage backend with blob-lease locking
Policy as code	OPA / Conftest with Rego v1
CI/CD	GitHub Actions
Cloud auth (CI)	Federated identity credentials on Microsoft Entra ID app
Compute	Azure Container App Job (Python 3.11)
Scheduler	Container Apps Job built-in cron trigger
AI summary	Microsoft Foundry hosting Phi-4-mini-instruct
Container registry	Azure Container Registry (Basic)
Storage	Blob Storage with HTTPS only, TLS 1.2, versioning
Email	Azure Communication Services Email
Runtime identity	User-assigned managed identity

Architecture summary

A scheduled cron trigger built into the Container App Job invokes a Python container monthly. The job authenticates with its user-assigned managed identity to read findings from Azure RBAC, Microsoft Defender for Cloud, and the Activity Log. It builds a CSV, uploads it to Blob Storage with a 90-day retention policy, generates a plain-English executive summary by calling the Foundry v1 OpenAI-compatible inference endpoint, and emails the report through Azure Communication Services. The same managed identity is used for every Azure call. No API keys exist anywhere in the system.

3. Demo Flow

Total runtime is roughly 12 minutes. Live, no slides. The demo is built around the 'two PR' pattern: one merged compliant change, one blocked policy violation. Both produce evidence, both end up in the run history.

Pre-demo setup

- Terminal open to the repo root, az CLI authenticated.
- Browser tabs: GitHub repo (Code tab), GitHub Actions tab, Azure portal on the resource group, Gmail inbox.
- VS Code open to terraform/identity.tf and policy/rbac_no_owner.rego.

Scene 1. Show the deployed system (1 minute)

Switch to the Azure portal on resource group rg-azure-access-review in eastus2. Walk through the deployed resources: the Container App Job, the Foundry resource with the Phi-4-mini-instruct deployment, the Blob storage account, the ACS resource and email service domain, the Log Analytics workspace. Talking point: 'Everything you see here was created by Terraform. There is no click-ops step.'

Scene 2. Trigger a manual run (2 minutes)

```
az containerapp job start \  
  --name azure-access-review-job \  
  --resource-group rg-azure-access-review
```

While it runs, switch to Log Analytics and tail the container logs live. Highlight the lines as they appear: 'Collected 23 total findings', 'Report uploaded to https://...azureaccessreviewrptq2lm.blob.core.windows.net/...', 'HTTP Request: POST .../openai/v1/chat/completions "HTTP/1.1 200 OK"' (this is the actual Phi-4-mini-instruct call), 'ACS email sent. Email sent to abdiemohamed.tech@gmail.com'. The job takes about 11 minutes end to end. Skip ahead by switching to the email.

Scene 3. Show the email arriving (2 minutes)

Open the Gmail thread. The email body contains the Phi-generated executive summary, three paragraphs covering overall posture, top issues, and recommended next steps. The attachment is the timestamped CSV with 23 findings sorted by severity. Open the CSV briefly to show severity distribution.

Scene 4. Walk the GitHub repository (3 minutes)

- **README.md**. Highlight the 'Why this is GRC engineering' section with explicit NIST control mappings (CM-2, CM-3, AU-2, AU-12, IA-2(8), IA-5, AC-3, AC-6, SC-7, SC-28). Talking point: 'The README is itself part of the audit evidence. It tells an auditor exactly which controls each design choice satisfies.'
- **policy/rbac_no_owner.rego**. Show the one-page policy file. The header comment names AC-6 and CMMC AC.L2-3.1.5. The body forbids Owner and User Access Administrator roles. Talking point: 'The control documentation IS the policy file. There is no separate Word doc that could drift from reality.'

- **terraform/identity.tf.** Show the role assignments: Reader, Security Reader, Monitoring Reader at subscription scope; Storage Blob Data Contributor on the report storage account; Cognitive Services OpenAI User on the Foundry resource; Contributor on the ACS resource. Six narrowly-scoped role assignments, each with a NIST control comment. Talking point: 'Least privilege at every layer. The job's identity has exactly the permissions it needs and nothing more.'
- **.github/workflows/terraform.yml.** Walk the pipeline order: checkout, federated OIDC auth, build and push image, fmt check, tfint, terraform plan with -input=false, OPA / Conftest gate, plan artifact upload, apply on push to main only. Talking point: 'The OPA gate runs before apply. A misconfiguration cannot reach Azure because the gate fails the build first.'

Scene 5. Live policy violation demo (3 minutes, the money shot)

Open a PR that adds a wildcard Owner role assignment somewhere in terraform/. Push the branch. Watch the GitHub Actions run fail at the OPA / Conftest step. The failure message names the rule by its NIST control reference:

```
Role assignment X grants forbidden role Owner
(violates NIST 800-53 AC-6 least privilege)
```

Try to merge the PR. Branch protection blocks it. Show that the audit evidence (the failed plan plus Conftest output) is still archived as a workflow artifact even on failure. Then close the PR. Talking point: 'Failed runs are auditable too. You don't lose the receipts when something goes wrong.'

Scene 6. Close (1 minute)

Show the pull request list with both PRs visible. One merged, one blocked. Both produced evidence. Both are in the run history.

Closing line: 'An auditor walks in tomorrow and asks how AC-6 is enforced. The answer is the failed PR plus the Rego file plus the policy run in the artifact. The YAML is the evidence. Compliance ships in the same pipeline as the system.'

4. Architecture Rationale: Why Each Choice

Each component in the stack was a deliberate choice over named alternatives. This section captures the reasoning so it can be defended in interviews and design reviews.

Why this overall architecture?

The system is designed around three independent layers that each address a distinct GRC failure mode:

- **Infrastructure layer (Terraform).** Every Azure resource is declared in code. This addresses 'undocumented infrastructure' and 'configuration drift between audit cycles.' The state is reproducible from git history.
- **Policy layer (OPA / Rego).** Security controls are encoded as machine-checkable rules that run before deploy. This addresses 'controls marked satisfied on paper but actually broken in production.' The policy file is the control documentation.
- **Evidence layer (CI artifacts + Activity Log).** Every change to the infrastructure produces an archived plan artifact. Every job execution produces a timestamped CSV in versioned storage plus log entries in Log Analytics. This addresses 'audit prep takes weeks of manual evidence collection.' Evidence is generated by construction, not by a human looking through screenshots.

Each layer is independently auditable. An auditor asking about any control gets the same answer pattern: the policy file, the Terraform code that the policy ran against, and the archived artifact proving the policy passed (or blocked the change) at a specific point in time.

Why Azure Container App Jobs (versus other compute)?

Considered alternatives:

- Azure Functions Consumption plan. Original choice. Hit zero default quota on the new subscription, blocking deploy.
- Azure Functions Premium plan. Removes the consumption quota wall but introduces a fixed monthly cost (~\$150/month) for what is functionally a once-monthly batch job.
- Azure Container Apps (the always-on service). Different from Container App Jobs; designed for HTTP request workloads, not scheduled batch.
- Azure Virtual Machines. Long-running compute. Patching, attestation, and persistent attack surface make this the wrong fit for a periodic workload.
- Azure Kubernetes Service. Operational overhead is wildly disproportionate to the workload.

Why Container App Jobs won:

- Uses a different quota pool from Functions Consumption (worked on this subscription where Functions did not).
- Built-in cron schedule trigger eliminates the need for a separate scheduler resource (Logic Apps, Event Grid, etc.).
- Scales to zero between executions. No idle compute cost.
- Container-based deployment lets us use any Python dependency without runtime restrictions.

- User-assigned managed identity supported natively, which is the basis of the auth story.

Why serverless here (versus a long-running service)?

The workload is fundamentally periodic: one execution per month per subscription, each lasting roughly 11 minutes. A long-running service would idle for 99.97% of the month while still incurring compute cost and attack surface. Serverless aligns the cost and security profile to the actual usage.

GRC-specific reasons serverless matters here:

- **No infrastructure to harden, patch, or attest.** The auditor scope shrinks from 'how do you secure your servers?' to 'what does the job do, and what can it access?' Both questions are answered by reading two files: `terraform/identity.tf` and `src/function/main.py`.
- **No persistent compute means no persistent attack surface.** The Container App Job replica exists for the duration of one report, then disappears. There is no long-lived host to be compromised.
- **Native Activity Log capture.** Every job execution, every Foundry inference call, every Blob PUT is logged automatically. Audit evidence is generated by Azure, not by the application code.
- **Pay-for-actual-use.** Twelve runs per year of a ten-minute container is functionally free. There is no idle infrastructure burning budget.

Why Microsoft Foundry + Phi-4-mini-instruct (versus other LLMs)?

Considered alternatives, all rejected:

- **Azure OpenAI gpt-4o-mini.** Original choice. Subscription has zero Azure OpenAI quota across all available regions. Quota request to Microsoft Support has no SLA on grant time. Blocking dependency.
- **Llama 3.1 8B Instruct via Foundry serverless.** Available on this subscription with no quota wall. Rejected because Meta-published models require a Marketplace subscription resource (adds the azapi provider dependency) and do not inherit Microsoft's federal attestations. Third-party model provenance creates a supply-chain audit obligation.
- **DeepSeek-V3 (Microsoft-direct on Foundry).** Cheap, OpenAI-compatible, no Marketplace dance. Rejected because multiple US federal and state entities have explicitly restricted DeepSeek-origin models on official systems regardless of who hosts them. Bad portfolio optics for any federal-aligned positioning.
- **GitHub Models (free tier).** Free, OpenAI-compatible, no quota. Rejected because Microsoft positions it as 'evaluation and prototyping' with no SLA, no data residency control, and no in-tenant audit logs. Acceptable for development; not defensible for a production GRC story.
- **Anthropic Claude via Azure Marketplace.** Strong model, available on Azure. Rejected for the same provenance reason as Llama: third-party model means an additional supply-chain attestation.

Why Phi-4-mini-instruct won:

- **Microsoft-trained, Microsoft-owned, Microsoft-hosted.** The model and the hosting service both live inside Azure AI Services' FedRAMP High (Commercial) and DoD IL5 (Government) attestations. There is no third-party provenance line to defend in audit.

- **Microsoft-direct billing.** No Marketplace subscription resource. Pure azurearm provider. Simpler Terraform, fewer moving parts.
- **Right-sized for the workload.** The task is summarizing 20-50 structured findings into a 3-paragraph executive summary. A 3.8B parameter model handles this comfortably. Using gpt-4o for it would cost 5-10x more for marginal quality lift.
- **Bypasses Azure OpenAI quota gate.** Microsoft-direct Foundry models use a separate quota pool with default capacity on new subscriptions.
- **v1 OpenAI-compatible inference endpoint.** The standard OpenAI Python SDK works against Foundry with two non-default settings (base_url and a token_provider callable). No vendor-specific client library.

Why this storage layer?

Considered: SQL DB, Cosmos DB, blob storage.

Why Blob Storage:

- Reports are immutable artifacts written once per execution and read by humans. Object storage is the right shape; database is overkill.
- Native versioning and lifecycle policies satisfy SC-28 (protection at rest) and SI-12 (information handling and retention) without code.
- Cheapest tier sufficient for the workload (~\$0.30/year).
- HTTPS-only, TLS 1.2 minimum, public network access disabled, all enforced by Rego before the resource can be created.

Why Azure Communication Services Email?

Considered: SendGrid, Office 365 SMTP, no email.

Why ACS Email:

- Native Azure service, billed through the same subscription. No third-party signup.
- Azure-managed sender domain ships immediately, no DNS verification step.
- Supports managed identity authentication. No connection strings or API keys.
- Pay-per-message pricing. Twelve messages per year cost roughly one cent.
- ACS itself holds FedRAMP High in Azure Commercial.

Why Terraform (versus Bicep, ARM, Pulumi)?

- **Bicep.** Azure-only. The two sibling repos (AWS, GCP) use Terraform; using Terraform here keeps the engineering pattern portable across the three clouds.
- **ARM JSON.** Verbose, hard to review in PR diffs.
- **Pulumi.** Strong for general-purpose programming languages but introduces dependency on a separate ecosystem; OPA / Rego policy gates are designed around Terraform plan output.

- **Terraform won** on cross-cloud portability, mature OPA tooling, and the existence of azurerm v4 with full Foundry coverage.

Why GitHub Actions for CI?

- Native federated identity support with Azure (no long-lived secrets in GitHub).
- Free for public repositories.
- Branch protection rules tie directly to required workflow checks.
- Plan artifacts retained as workflow artifacts for 30 days; doubles as audit evidence storage at no extra cost.

5. NIST 800-53 Control Families Hit

Five control families from NIST 800-53 Rev 5 are explicitly satisfied by the design of this project. Each family has at least one specific control implemented, and most have multiple. The cross-mappings to CMMC L2 are in the next section.

Family	Description	How this project hits it
AC	Access Control	Managed identity replaces all credential storage. OPA blocks Owner role assignments via Terraform. Each role assignment is scoped to one resource where possible. Storage public network access disabled.
AU	Audit and Accountability	Activity Log captures every Azure API call. Container App Job logs land in Log Analytics. Every CI run produces an archived plan artifact retained for 30 days. Failed PRs also produce artifacts.
CM	Configuration Management	All Azure resources declared in Terraform. OPA gate runs on every PR; misconfigurations cannot reach Azure. State is reproducible. No click-ops.
IA	Identification and Authentication	Federated identity credentials between GitHub Actions and Microsoft Entra ID. User-assigned managed identity for runtime. No long-lived credentials anywhere in the system.
SC	System and Communications Protection	Storage HTTPS-only, TLS 1.2, public access disabled (SC-7). Storage versioning enforced via OPA (SC-28 protection at rest). Foundry custom subdomain for Entra ID auth.

6. How the Solution Satisfies Specific Controls

Each control below has a citable enforcement mechanism in the project. The 'How it is satisfied' column is what an auditor needs: the specific file or behavior that proves the control is operational. This is the GRC engineering answer to 'show me your evidence.'

Control	Title	How it is satisfied (the audit answer)
AC-3	Access Enforcement	Mechanism: Rego policy blocks storage accounts that allow public access. Evidence file: policy/storage_secure.rego. Enforcement point: CI runs Conftest before terraform apply; failing the rule blocks the merge. Bonus: ACS Email sender authenticates only via managed identity, no anonymous send path exists.
AC-6	Least Privilege	Mechanism (1): Rego policy blocks any Terraform resource that grants Owner or User Access Administrator. Evidence file: policy/rbac_no_owner.rego. Mechanism (2): Each role assignment in terraform/identity.tf is scoped to one resource where possible. Reader, Security Reader, Monitoring Reader at sub scope (read-only); Storage Blob Data Contributor scoped to one storage account; Cognitive Services OpenAI User scoped to one Foundry resource; Contributor scoped to one ACS resource. Six narrowly-scoped role assignments, no wildcards.

Control	Title	How it is satisfied (the audit answer)
AU-2	Audit Events	Mechanism: Azure Activity Log captures every control-plane operation by default. Mechanism: Container App Job stdout/stderr is captured to Log Analytics workspace 'azure-access-review-logs' (configured in terraform/function.tf). Evidence: 30-day default retention on Activity Log, configurable in Log Analytics.
AU-12	Audit Generation	Mechanism: GitHub Actions archives every Terraform plan as a workflow artifact, retained for 30 days. Critical: plan artifacts are retained on BOTH passing and failing runs. The blocked PR's evidence is preserved. Evidence path: .github/workflows/terraform.yml lines defining 'Upload plan artifact' step.
CM-2	Baseline Configuration	Mechanism: All Azure resources declared in terraform/*.tf. State: stored in versioned Azure Blob Storage (azurerm backend with use_azuread_auth, blob versioning enabled). Reproducibility: a fresh checkout plus terraform apply recreates the exact deployed state. No drift mechanism: click-ops changes show up in the next plan as resource diffs, surfacing drift on the next run.
CM-3	Configuration Change Control	Mechanism: .github/workflows/terraform.yml runs the OPA / Conftest gate on every PR. Enforcement: branch protection rules require the workflow check to pass before merge. Apply gate: apply only runs on push to main, after the same checks pass again. Net effect: a non-compliant change cannot reach production without a documented branch-protection bypass.
IA-2(8)	Replay-Resistant Authentication	Mechanism: federated identity credentials on a Microsoft Entra ID app. Token flow: GitHub Actions presents an OIDC token; Entra ID exchanges it for a short-lived access token. Restriction: federated credential 'subject' claim binds to a specific repo and branch, e.g. repo:a bdie-grcengineer/terraform-azure-automated-access-review:ref:refs/heads/main. Net effect: tokens are not reusable outside the specific workflow run that minted them.
IA-5	Authenticator Management	Mechanism: no long-lived credentials exist anywhere in the system. Runtime auth: user-assigned managed identity attached to the Container App Job (terraform/function.tf). ACS Email auth: managed identity, not connection string. Foundry inference auth: managed identity, not API key. Storage auth: managed identity, not SAS token. Foundry resource itself: local_auth_enabled = false (terraform/foundry.tf), forcing Entra ID auth and disabling API keys at the resource level.
SC-7	Boundary Protection	Mechanism: Rego policy enforces HTTPS-only traffic, TLS 1.2 minimum, and disabled public network access on every Terraform-managed storage account. Evidence file: policy/storage_secure.rego. Reinforcement: the report storage account in terraform/storage.tf has these settings hard-coded; the Rego policy guards against future drift.

Control	Title	How it is satisfied (the audit answer)
SC-28	Protection of Information at Rest	Mechanism: Rego policy enforces blob versioning on every storage account. Versioning protects audit evidence from accidental or malicious overwrite, which is the practical SC-28 risk for object storage. Evidence file: policy/storage_encryption.rego. Encryption itself: automatic in Azure with Microsoft-managed keys; no explicit configuration needed for default coverage. Customer-managed keys are a follow-on enhancement.

7. CMMC Level 2 Cross-Mapping

CMMC L2 controls are derived from NIST 800-171 which itself draws from NIST 800-53. The table below maps each CMMC L2 practice this project satisfies back to its NIST 800-53 ancestor and to the project artifact that enforces it.

CMMC L2	Practice	NIST 800-53 ancestor	Enforced by
AC.L2-3.1.3	Control CUI flow	AC-3, SC-7	policy/storage_secure.rego
AC.L2-3.1.5	Apply principle of least privilege	AC-6	policy/rbac_no_owner.rego
AU.L2-3.3.1	Create and retain audit logs	AU-2, AU-12	Activity Log + Log Analytics + GitHub Actions artifacts
CM.L2-3.4.2	Establish and enforce config settings	CM-2, CM-3	Terraform + OPA gate
IA.L2-3.5.3	Use multifactor or replay-resistant auth	IA-2(8)	Federated identity credentials, managed identity
SC.L2-3.13.1 6	Protect confidentiality of CUI at rest	SC-28	policy/storage_encryption.rego

8. Service Stack and Federal Availability

Every service used in this project carries a FedRAMP High authorization in Azure Commercial. Two services have caveats for Azure Government and are flagged below. Both have documented fallbacks that are themselves FedRAMP High and DoD IL5.

Service	FedRAMP High (Commercial)	Azure Government (DoD IL5)
Azure Storage (Blob)	Yes	Yes
Azure Container Registry	Yes	Yes
Azure Container Apps (Jobs)	Yes	Yes
Microsoft Entra ID	Yes	Yes
Microsoft Defender for Cloud	Yes	Yes
Azure Monitor / Log Analytics	Yes	Yes
Azure Resource Graph	Yes	Yes
Azure Key Vault	Yes	Yes
Microsoft Foundry (AIServices) + Phi-4-mini-instruct	Yes	Verify per region. Fallback: Azure OpenAI gpt-4o-mini, GA in Azure Gov, FedRAMP High and IL5.

Service	FedRAMP High (Commercial)	Azure Government (DoD IL5)
Azure Communication Services Email	Yes	Verify status. Fallback: SendGrid via Azure Marketplace or in-tenant SMTP relay.

Defensible framing: The architecture is portable across Azure Commercial and Azure Government. The two flagged services have documented swap paths that preserve the FedRAMP High and DoD IL5 posture. Compute, storage, identity, and audit logs all carry over directly without changes.

9. Implementation Friction Encountered

The build hit three categories of friction. This is the honest answer to the question 'how hard was it?'

Category 1: Quota gating (subscription-level feature gates)

- Azure OpenAI quota = 0 on a new subscription. Forced pivot to Microsoft Foundry + Phi-4-mini-instruct (which uses a separate quota pool).
- Functions Consumption quota = 0 on a new subscription. Forced pivot from Azure Functions to Azure Container App Jobs.
- ACR Tasks blocked at the subscription level. Forced moving image build out of Terraform's local-exec into the GitHub Actions runner.

These three share a root cause: Microsoft restricts certain compute and AI features on new subscriptions to throttle abuse. Enterprise subscriptions do not see them.

Category 2: RBAC granularity (would hit any subscription)

- Storage Blob Data Owner is a separate role from Owner. Owner at subscription scope manages storage accounts but cannot read their blobs. Required explicit data-plane grant for terraform init to succeed against the Azure Storage backend.
- ACS Email rejected the managed identity until an explicit Contributor role was granted on the ACS resource. Pre-existing gap in the role set; manifested as 'Denied by the resource provider' which looked like a token problem.

Category 3: API subtleties

- Foundry v1 inference uses a different audience claim ('https://ai.azure.com/.default') than legacy Azure OpenAI ('https://cognitiveservices.azure.com/.default'). Wrong audience produces a 401 that looks like RBAC.
- Cognitive Services accounts require a custom subdomain for Entra ID auth to work at all. Without it, only API key auth is accepted, which kills the managed-identity story.

Talking-point reframing

Lead-in line: 'None of it was Azure being broken. All of it was Azure being more granular than AWS or GCP about who can do what to which resource.' This reframes 'I hit walls' as 'I learned how Azure's permission model works at the data-plane level,' which is exactly the skill an interviewer wants to see in a

GRC engineer.

10. Cost Profile

As deployed, the system costs roughly two to five US dollars per year, all in. ACR Basic is the dominant cost. The LLM is rounding error.

Component	Annual cost (deployed)
Phi-4-mini-instruct inference (12 runs/year, ~25K tokens each)	~\$0.10
Container App Job execution (12 runs/year, ~10 min, scales to zero)	~\$0.10
Azure Container Registry (Basic SKU, single image)	~\$5.00
Blob Storage (reports + state, LRS, well under 1 GB)	~\$0.30
Log Analytics (job logs, low volume)	~\$0.50
ACS Email (12 messages/year)	~\$0.01
TOTAL	~\$2 to \$5 per year

Federal-scale projection

A federal agency would likely run this against many subscriptions and possibly daily rather than monthly. Scaling assumptions: 50 subscriptions, daily run instead of monthly (~30x), 10x more findings per run.

That puts annual cost in the **one thousand to three thousand US dollars range per environment**. For comparison: a typical commercial CSPM tool runs fifty thousand to two hundred thousand dollars per year per cloud per environment. The 'control is the code' approach is roughly two orders of magnitude cheaper than buying a dashboard.

The deeper cost story is engineering time saved. A traditional GRC team manually evidencing AC-3, AC-6, SC-7, SC-28 compliance across that many subscriptions burns roughly a person-month per audit cycle. This automation completes the same evidence in 11 minutes per subscription, generated continuously.

11. Notable Design Decisions

LLM choice: Phi-4-mini-instruct on Microsoft Foundry

Selected over Llama 3.1 8B and DeepSeek-V3. Phi is Microsoft-trained, Microsoft-owned, and Microsoft-hosted, sitting inside Azure AI Services' FedRAMP High and DoD IL5 attestations. Selecting a Microsoft-published model removes third-party model provenance from any future supply-chain audit. Llama was rejected because Meta-published models do not inherit Microsoft's federal attestations. DeepSeek was rejected because multiple US federal and state entities have explicitly restricted DeepSeek-origin models on official systems.

Image build decoupled from Terraform

The container image is built and pushed to Azure Container Registry by GitHub Actions before terraform apply. Terraform only references the pre-built image. This is the production pattern: CI builds artifacts, CD applies infrastructure, neither knows about the other. Side benefit: Terraform replans never accidentally rebuild the image.

User-assigned managed identity, not system-assigned

The job uses a user-assigned managed identity rather than system-assigned so the identity exists before the Container App Job resource is created. RBAC role assignments target the managed identity directly, which makes Terraform apply ordering deterministic and avoids race conditions on first deploy.

Federated identity for CI authentication

GitHub Actions presents an OIDC token; an Entra ID app registration exchanges it for a short-lived Azure access token. The federated credential subject claim restricts to a specific repo and ref. No long-lived Azure credentials are stored in GitHub Secrets. Maps directly to NIST 800-53 IA-2(8) replay-resistant authentication.

12. Quick-Reference Talking Points

Short, defensible answers for common interview prompts.

'Why Azure for this versus AWS or GCP?'

Three native Azure features made this design clean: federated identity credentials between Entra ID and GitHub Actions, Container App Jobs with built-in cron triggers (no separate scheduler resource), and Microsoft Foundry hosting Microsoft-owned models inside Azure AI Services' compliance perimeter. The same pattern works on AWS (with Bedrock + Lambda + EventBridge) and GCP (with Vertex AI + Cloud Functions + Cloud Scheduler), and I have parallel implementations in both. The cloud is pluggable; the GRC engineering pattern is what matters.

'How does this satisfy FedRAMP continuous monitoring?'

FedRAMP continuous monitoring requires recurring evidence of control effectiveness. This system produces that evidence by construction: scheduled job execution with timestamped CSV outputs, archived in versioned blob storage with a 90-day retention lifecycle, plus the GitHub Actions plan artifacts on every config change. An auditor sampling any month sees both the resource-state evidence (the CSV) and the change-control evidence (the plan artifact) for that month.

'What controls would you add next?'

Three obvious extensions: AC-2 (Account Management) by polling Entra ID for stale users and orphaned service principals; AC-17 (Remote Access) by inventorying public-facing storage accounts and Front Door endpoints; and AU-6 (Audit Review and Reporting) by adding anomaly detection over the Activity Log signals already collected. Each would be a new Rego policy plus a new findings collector module, no architectural changes.

'How would you respond to a control that fails the OPA gate?'

The gate names the rule and the resource address in its failure message. First step is to read the policy file to understand what the rule requires. Second is to either fix the Terraform to satisfy the rule, or if the rule itself is wrong, propose a policy update through the same PR review process. Either path leaves a paper trail. The control that blocked the change is documented and version-controlled.

End of brief. Repository: github.com/abdie-grcengineer/terraform-azure-automated-access-review. For full design rationale see [docs/design-decisions.md](#). For implementation gotchas see [docs/lessons-learned.md](#).